Student Meal Planner

Database and Data Access Strategy: ADO.NET and SQL Server

A Technical Research Report for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

Introduction

Grubs4Scrubs needs a database. That part was obvious from day one. The less obvious part was how to talk to it. Most ASP.NET Core tutorials jump straight to Entity Framework Core, which handles SQL generation, migrations, and object mapping automatically. But for this project, EF Core wasn't an option. The course requires raw ADO.NET, meaning I'd be writing SQL by hand and mapping query results to C# objects myself.

That constraint changed a lot of decisions. I had to think about how to structure tables without relying on EF's migration tools, how to keep raw SQL organized instead of scattered across controllers, and how to make the data access layer testable even without an ORM doing the heavy lifting. This report covers those decisions: why SQL Server, how ADO.NET works compared to EF Core, how the repository pattern keeps things clean, and how the Recipes and Users tables were designed. It fits under Implementation in the portfolio because it's about the actual technical choices made during development.

Main Research Question

What is the best approach to database design and data access for Grubs4Scrubs using raw ADO.NET and SQL Server?

Sub-Questions, Methods, and Approach

I broke the main question into four parts. Most of the research was library-based (reading official docs) combined with hands-on experimentation in the actual project. There's no point theorizing about ADO.NET if you haven't tried opening a SqlConnection yourself.

Why SQL Server Over Other Databases?

Sub-question: *Why is SQL Server a good fit for Grubs4Scrubs, and what are the alternatives?*

Method: Available product analysis, comparing database options for small .NET projects.

I looked at the databases commonly used with ASP.NET Core: SQL Server, PostgreSQL, SQLite, and MySQL. The comparison focused on what actually matters for a student project, things like tooling support in VS Code, how well the database integrates with .NET's built-in libraries, and whether the setup process would eat into development time. I also considered what my coach and the course materials assumed I'd be using.

How Does ADO.NET Compare to Entity Framework Core?

Sub-question: *What are the practical differences between using raw ADO.NET and Entity Framework Core for data access?*

Method: Library research using Microsoft Learn documentation and community comparisons.

I read Microsoft's documentation on both ADO.NET and EF Core, then looked at blog posts from .NET developers who've used both. The goal wasn't to decide which is "better" in the abstract (that depends on the project), but to understand what I'd gain and lose by using ADO.NET specifically. I also looked at what patterns exist to keep raw SQL manageable, since that's the biggest risk with ADO.NET.

How Does the Repository Pattern Work With ADO.NET?

Sub-question: *How should data access code be organized using the repository pattern when there's no ORM?*

Method: Library research and best good and bad practices analysis.

Most repository pattern examples online assume EF Core. I had to find examples that use SqlConnection and SqlCommand directly. I looked at Microsoft's own guidance on the repository pattern, then studied how open-source projects implement it without an ORM. The key question was: does the pattern still make sense when you're writing raw SQL, or does it just add boilerplate?

What Should the Database Schema Look Like?

Sub-question: *How should the Recipes and Users tables be designed to support Grubs4Scrubs' features?*

Method: Library research on database normalization and best good and bad practices for table design.

I reviewed basic normalization rules (1NF through 3NF) and looked at how similar meal planning apps structure their data. Then I designed the tables based on what the frontend actually needs: recipe cards with titles, images, tags, prep times, and budget estimates, plus user accounts with email/password and optional Google login.

Results: Sources Reviewed

The following sources shaped the decisions documented here. Each one addressed a different part of the data access problem.

Microsoft Learn: ADO.NET Overview

Microsoft's documentation on ADO.NET covers the core classes: SqlConnection for connecting to the database, SqlCommand for executing queries, and SqlDataReader for reading results row by row. The docs are clear that ADO.NET gives you full control over the SQL being executed, which means better performance tuning but more code to write. The key objects (Connection, Command, Reader) follow IDisposable, so they need `using` statements to avoid connection leaks.

Microsoft Learn: Entity Framework Core vs. ADO.NET

Microsoft's comparison documentation doesn't explicitly tell you to pick one over the other, but it's clear about the tradeoffs. EF Core handles SQL generation, change tracking, and migrations automatically. ADO.NET gives you raw SQL control and slightly better performance for simple queries, but you're responsible for mapping results to objects yourself. For projects where the SQL is straightforward (CRUD operations on a few tables), ADO.NET doesn't add much complexity. For projects with dozens of tables and complex relationships, EF Core saves significant time.

Microsoft Learn: Repository Pattern

The Microsoft docs describe the repository pattern as a way to abstract data access behind an interface. The controller (or service) calls `IRecipeRepository.GetAll()` and gets a list of Recipe objects back. It doesn't know or care whether the implementation uses EF Core, ADO.NET, or a CSV file. This abstraction makes it possible to swap out the data access layer without changing business logic, and it makes unit testing easier because you can mock the repository interface.

SQL Server Documentation: Table Design Best Practices

Microsoft's SQL Server documentation covers table design principles including choosing appropriate data types, setting nullable vs. non-nullable columns, and using identity columns for auto-incrementing primary keys. The docs recommend using `NVARCHAR` over `VARCHAR` for text that might contain Unicode characters, and using `DATETIME2` over `DATETIME` for better precision and range. For small tables (under a few million rows), the performance difference between these choices is negligible, but picking the right types from the start avoids painful migrations later.

Stack Overflow and Community Discussions on ADO.NET Patterns

Several highly-voted Stack Overflow threads discuss how to organize ADO.NET code in modern .NET projects. The consensus is that wrapping SqlConnection/SqlCommand in repository classes is the standard approach. Common advice includes: always use

parameterized queries (never string concatenation) to prevent SQL injection, use `using` statements for all IDisposable objects, and keep the mapping logic (SqlDataReader to C# object) in a private helper method to avoid repeating it in every query.

Conclusion: Sub-Questions

Each sub-question pointed toward a specific part of the data access strategy. Here's what I settled on and why.

SQL Server Choice, Answered

SQL Server is the right fit for Grubs4Scrubs for practical reasons more than technical ones. The .NET ecosystem has first-class support for SQL Server through Microsoft.Data.SqlClient. The VS Code mssql extension provides a solid query editor and connection manager without needing full SSMS. The course materials and coach guidance assume SQL Server. PostgreSQL would work fine technically, but switching would mean fighting tooling and documentation that assumes SQL Server. For a student project with a deadline, that's not a tradeoff worth making.

ADO.NET vs. EF Core, Answered

ADO.NET is required for this project, but even setting that aside, it's a reasonable choice for Grubs4Scrubs. The app has two main tables (Recipes and Users) with straightforward CRUD operations. There are no complex joins or relationship graphs that would make EF Core's navigation properties valuable. Writing the SQL by hand takes more lines of code, but each query is explicit and easy to debug. The main downside is that there are no automatic migrations, so schema changes require manual SQL scripts. For a project this size, that's manageable.

Repository Pattern With ADO.NET, Answered

The repository pattern works well with ADO.NET. Each repository class takes an IConfiguration in its constructor (to read the connection string), opens a SqlConnection for each operation, executes a parameterized SqlCommand, and maps the results using a private helper method. The interface (IRecipeRepository) defines the contract, the implementation (RecipeRepository) handles the SQL. The service layer calls the interface, never the implementation directly. This keeps SQL completely out of the controllers and services, which is the whole point.

Database Schema, Answered

The Recipes table has columns for Id (int, identity, primary key), Title (nvarchar, not null), Description (nvarchar), PrepTime and CookTime (int, minutes), Servings (int), Tag (nvarchar), ImageUrl (nvarchar), EstimatedBudget (decimal), Category (nvarchar), Ingredients and Instructions (nvarchar max, stored as JSON strings), UserId (int, nullable foreign key to Users), nutrition fields (Calories, Protein, Carbs, Fats as int), Tips (nvarchar), and CreatedAt (datetime, default to current UTC time). The Users table has Id, Email, PasswordHash, UserName, GoogleId (nullable, for optional Google OAuth), and CreatedAt. The ShoppingItems table has Id, Name, Quantity (nvarchar, because users type things like "2 cans" or "500g"), Price (decimal), IsChecked (bit, for the checkbox state), and UserId (nullable, for future per-user filtering). All three tables use identity columns for auto-incrementing IDs.

Conclusion: Main Question

The best approach for Grubs4Scrubs is SQL Server accessed through raw ADO.NET, with all data access code wrapped in repository classes that implement interfaces. The connection string lives in appsettings.json, repositories read it through IConfiguration, and ASP.NET Core's dependency injection wires everything together at runtime. Each repository handles one table's CRUD operations, keeps SQL in parameterized queries, and maps results through a private helper method.

For Grubs4Scrubs specifically, this means the RecipeRepository contains all recipe-related SQL in one place. Adding a new query (like filtering recipes by tag or searching by title) means adding one method to the interface and one implementation. The service layer validates input before calling the repository, and the controller stays thin. The approach is more verbose than EF Core, but it's explicit, debuggable, and meets the course requirement. It also forced me to actually understand SQL instead of relying on an ORM to generate it, which turned out to be a useful learning experience for the portfolio.

Summary

Grubs4Scrubs uses SQL Server with raw ADO.NET for all data access. The repository pattern wraps SqlConnection and SqlCommand behind interfaces (IRecipeRepository, IUserRepository, IShoppingItemRepository), keeping SQL out of the service and controller layers. The database has three tables (Recipes, Users, and ShoppingItems) designed around what the frontend actually needs. This approach meets the course's no-EF-Core requirement while keeping the codebase organized and testable through dependency injection.

References

All references are formatted in APA 7th edition style.

Microsoft. (2024). *ADO.NET overview.* Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ado-net-overview

Microsoft. (2024). *Compare EF Core & EF6.* Microsoft Learn. https://learn.microsoft.com/en-us/ef/efcore-and-ef6/

Microsoft. (2024). *Design the infrastructure persistence layer.* Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design

Microsoft. (2024). *SQL Server documentation: Table design.* Microsoft Learn. https://learn.microsoft.com/en-us/sql/relational-databases/tables/tables

Microsoft. (2024). *Dependency injection in ASP.NET Core.* Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection